

1. MIT Flow-Matching and Diffusion Course

[[MIT Course 6.S184: Generative AI with Stochastic Differential Equations](#)]

1.1 Introduction

Key Idea 1 (Objects as Vectors)

We identify the objects being generated as vectors $z \in \mathbb{R}^d$.

Key Idea 2 (Generation as Sampling)

Generating an object z is modeled as sampling from the data distribution $z \sim p_{\text{data}}$.

Key Idea 3 (Dataset)

A dataset consists of a finite number of samples $z_1, \dots, z_N \sim p_{\text{data}}$.

Key Idea 4 (Guided Generation)

Guided generation involves sampling from $z \sim p_{\text{data}}(\cdot|y)$, where y is a conditioning variable.

- Generative Models: Abstractly speaking, a generative model is an algorithm that returns samples from $z \sim p_{\text{data}}$ (or at least approximately). If p_{data} is the distribution of images of dogs, this algorithm would return random images of dogs [将生成object视作对于 objects' distribution的建模, eg: 在 $\mathbb{R}^d$ 空间中要生成dog representation vector, 就要在 $\mathbb{R}^d$ 空间中建模dogs' distrubion, 建模即明确some vector属于dogs的likelihood比较大, other vector属于dogs的likelihood比较小, 可以形式化为 $[z, p(z)]^T \in [\mathbb{R}^d, [0, 1]^T]$ (d+1) 维]
- Model Assumption: p_{data} 与 $p_{\text{data}}(\cdot|y)$ exit yet unavalible, 但是avalible dataset $\mathcal{D} = \{z_i\}_{i=1}^N$ 为distribution的有限采样, 因此将生成问题形式化为从离散采样的数据样本中对数据分布进行建模, 然后从该分布中采样即可以得到生成对象。(因为分布是连续的, 采样点不一定来自于数据集, 此时生成对象则不在数据集中, 但仍属于该分布目标, 为“creative generation”)

1.2 Flow Models

1.2.1 ODEs(ordinary differential equations)

1.2.1.1 Related Definition:

- Trajectory Def.: a function of form: $X : [0, 1] \rightarrow \mathbb{R}^d, t \mapsto X_t$, 该函数将时间映射到空间中的某一点, 表: 某时刻下对象在空间中的位置矢量
- Vector field Def.: a function of form: $u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, (x, t) \mapsto u_t(x)$, 该函数表: 空间中某点在某时刻的速度矢量
- ODEs是描述对象在vector field中运动的方程:
 - $\frac{d}{dt}X_t = u_t(X_t)$: ODE imposes a condition on a trajectory: we want a trajectory X that “follows along the lines” of the vector field
 - $X_0 = x_0$: initial conditions indicate starting at the point x_0

对于已知ODE与initial condition的方程, ODE ‘s solution is one trajectory, 但现在研究扩展到空间中的任意一点, 即starting at each point in space, 想要知道在vector field μ 中, 空间中任意点是如何运动的, 因此我们引出flow来描述这种空间中所有点的运动。

- Flow Def.: a function of form: $\psi : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, (x_0, t) \mapsto \psi_t(x_0)$
 - 对比trajectory, flow就是输入多了 $x_0 \in \mathbb{R}^d$ 的函数, 表: 某时刻下某对象在空间中的位置矢量。可以认为是所有trajectory的“集合”
 - $\frac{d}{dt}\psi_t(x) = u_t(\psi_t(x))$ --Flow ODE; $\psi_0(x) = x$ --Flow initial conditions
 - Flow is the solution of ODE without initial condition, 是用来描述整个空间点的运动, 而非某一特定start point

1.2.1.2 Flow existence and uniqueness

Theorem 3 (Flow existence and uniqueness)

If $u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is continuously differentiable with a bounded derivative, then the ODE in (2) has a unique solution given by a flow ψ_t . In this case, ψ_t is a **diffeomorphism** for all t , i.e. ψ_t is continuously differentiable with a continuously differentiable inverse ψ_t^{-1} .

- Vector field u 连续可微且有界, 则对应解的 ψ 存在且唯一, 并且连续可微存在唯一逆 ψ^{-1} 。
在Machine Learning中我们默认该条件成立, not care

1.2.1.3 Simulating an ODE

- 部分ODE存在存在封闭解, 即可以写出 ψ 的表达式, 但多数复杂情况只能通过numerical methods to simulate ODEs近似求解trajectory (flow given initial condition), 常用的方法如下:

and most intuitive methods is the **Euler method**. In the Euler method, we initialize with $X_0 = x_0$ and update via

$$X_{t+h} = X_t + hu_t(X_t) \quad (t = 0, h, 2h, 3h, \dots, 1-h) \quad (4)$$

where $h = n^{-1} > 0$ is the **step size** and $n \in \mathbb{N}$ is the number of simulation steps. For this class, the Euler method will be good enough. To give you a taste of a more complex method, let us consider **Heun's method** defined via the update rule

$$\begin{aligned} X'_{t+h} &= X_t + hu_t(X_t) && \blacktriangleright \text{initial guess of new state (same as Euler step)} \\ X_{t+h} &= X_t + \frac{h}{2}(u_t(X_t) + u_{t+h}(X'_{t+h})) && \blacktriangleright \text{update with average } u \text{ at current and guessed state} \end{aligned}$$

Intuitively, Heun's method is as follows: it takes a first guess X'_{t+h} of what the next step could be but corrects the direction initially taken via an updated guess.

1.2.2 Why ODEs and Flow ?



1. 由1.1, 将generation形式化为“从已有采样的有限数据集建模distribution p_{data} 再采样生成”的数学问题。
2. 如何model distribution? 直接地, 已有的dataset samples只能将概率空间建模为离散的"尖峰"分布, 这样采样生成也就只会“背出数据集的答案”。
3. 进一步我们希望神经网络可以学习有意义的连续分布。直接地, 学习 $p_\theta(\cdot)$, 输入 $x \in \mathbb{R}^d$, 输出 $\hat{p}(x)$, 但是因为 p_{data} 不可得, 没有合适的 $p_{data}(x)$ 标签, 并且也无法约束 $\sum_{x \in \mathbb{R}^d} p_{data}(x) = 1$ 。
4. 因此, 联想ODE's flow用来描述从 $t = 0$ 到 $t = 1$ 空间中所有点的运动, 所以也用ODE来描述 $t = 0$ 到 $t = 1$ 概率空间所有点的运动(但其中每个start point x 都是有"概率密度" $p_{init}(x)$, 我们只需要跟踪flow就可以得到 $t = 1$ 移动到 $z \in \mathbb{R}^d$, 则 $p(z) = \sum_{X_{t=1}=z} p_{init}(x)$, 即描述 $p_{init} \rightarrow p_{data}$ 概率分布的变化。这给我们一个转换的思路, 可以从确定的 p_{init} 出发, 让网络拟合学习概率空间的vector field u , 则也就可以"通过numerical method得到" p_{data}

1.2.3 Flow model Formulation

1.2.3.1 声明变量:

- Transport time t from 0 to 1, $t \in [0, 1]$
- manual designed p_{init} , unavailable p_{data} (Remark: Here $p_{init} := \mathcal{N}(0, I)$ always, 但理论上可有不同选择)
- sample x from initial distribution $x \sim p_{init}$
- z in dataset $\mathcal{D}$ is sampled from data distribution $z \sim p_{data}$
- Trajectory X_t , where initial condition $X_0 = x$

1.2.3.2 Flow model's distribution ODE

- $\frac{d}{dt}X_t = u_t^\theta(X_t)$ ODE: 神经网络拟合的是vector field, not the flow
- $X_0 = x, x \sim p_{init} \quad (\Leftrightarrow X_0 \sim p_{init})$ flow initial conditions
- Remark:
 - 上述两式相比于前边的ODE不同点: 用神经网络拟合vector field + initial condition 多了 $p_{init}(x)$
 - 从 p_{init} 采样时取到 $p_{init}(x)$ 较高的start point, 经transport之后也依旧是“概率密度” $\geq p_{init}(x)$ 的end point

Algorithm 1 Sampling from a Flow Model with Euler method

Require: Neural network vector field u_t^θ , number of steps n

```
1: Set  $t = 0$ 
2: Set step size  $h = \frac{1}{n}$ 
3: Draw a sample  $X_0 \sim p_{init}$ 
4: for  $i = 1, \dots, n$  do
5:    $X_{t+h} = X_t + hu_t^\theta(X_t)$ 
6:   Update  $t \leftarrow t + h$ 
7: end for
8: return  $X_1$ 
```

1.3 Diffusion Model

1.3.1 SDEs

Stochastic differential equations (SDEs) extend the deterministic trajectories from ODEs with **stochastic** trajectories. A stochastic trajectory is commonly called a **stochastic process** $(X_t)_{0 \leq t \leq 1}$ and is given by

$$X_t \text{ is a random variable for every } 0 \leq t \leq 1$$
$$X : [0, 1] \rightarrow \mathbb{R}^d, \quad t \mapsto X_t \text{ is a random trajectory for every draw of } X$$

In particular, when we simulate the same stochastic process twice, we might get different outcomes because the dynamics are designed to be random.

1.3.1.1 Brownian Motion(布朗运动)

可以“简单认为” SDEs = ODEs + Brownian Motion (Stochastic source)

- 详细的Brownian Motion见下(not care much)

Brownian Motion. SDEs are constructed via a **Brownian motion** - a fundamental stochastic process that came out of the study physical diffusion processes. You can think of a Brownian motion as a continuous random walk. Let us define it: A **Brownian motion** $W = (W_t)_{0 \leq t \leq 1}$ is a stochastic process such that $W_0 = 0$, the trajectories $t \mapsto W_t$ are continuous, and the following two conditions hold:

1. **Normal increments:** $W_t - W_s \sim \mathcal{N}(0, (t-s)I_d)$ for all $0 \leq s < t$, i.e. increments have a Gaussian distribution with variance increasing linearly in time (I_d is the identity matrix).
2. **Independent increments:** For any $0 \leq t_0 < t_1 < \dots < t_n = 1$, the increments $W_{t_1} - W_{t_0}, \dots, W_{t_n} - W_{t_{n-1}}$ are independent random variables.

Brownian motion is also called a **Wiener process**, which is why we denote it with a "W".¹ We can easily simulate a Brownian motion approximately with step size $h > 0$ by setting $W_0 = 0$ and updating

$$W_{t+h} = W_t + \sqrt{h}\epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I_d) \quad (t = 0, h, 2h, \dots, 1-h) \quad (5)$$

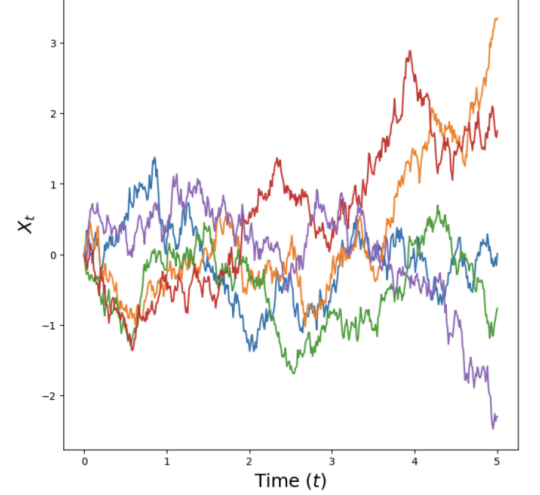


Figure 2: Sample trajectories of a Brownian motion W_t in dimension $d = 1$ simulated using Equation (5).

1.3.1.2 From ODEs to SDEs

- 将ODE从微分方程改写为 Δ 增量表达的等价形式

$$\begin{aligned} \frac{d}{dt}X_t &= u_t(X_t) && \blacktriangleright \text{expression via derivatives} \\ \Leftrightarrow \quad \frac{1}{h}(X_{t+h} - X_t) &= u_t(X_t) + R_t(h) \\ \Leftrightarrow \quad X_{t+h} &= X_t + hu_t(X_t) + hR_t(h) && \blacktriangleright \text{expression via infinitesimal updates} \end{aligned}$$

where $R_t(h)$ describes a negligible function for small h , i.e. such that $\lim_{h \rightarrow 0} R_t(h) = 0$, and in (i) we sir

- 再增加Brownian Motion stochastic bias

$$X_{t+h} = X_t + \underbrace{hu_t(X_t)}_{\text{deterministic}} + \underbrace{\sigma_t(W_{t+h} - W_t)}_{\text{stochastic}} + \underbrace{hR_t(h)}_{\text{error term}} \quad (6)$$

where $\sigma_t \geq 0$ describes the **diffusion coefficient** and $R_t(h)$ describes a stochastic error term such that the standard deviation $\mathbb{E}[\|R_t(h)\|^2]^{1/2} \rightarrow 0$ goes to zero for $h \rightarrow 0$. The above describes a **stochastic differential equation (SDE)**. It is common to denote it in the following symbolic notation:

$$dX_t = u_t(X_t)dt + \sigma_t dW_t \quad \blacktriangleright \text{SDE} \quad (7a)$$

$$X_0 = x_0 \quad \blacktriangleright \text{initial condition} \quad (7b)$$

- SDE 解的存在性与唯一性同ODE

Theorem 5 (SDE Solution Existence and Uniqueness)

If $u : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is continuously differentiable with a bounded derivative and σ_t is continuous, then the SDE in (7) has a solution given by the unique stochastic process $(X_t)_{0 \leq t \leq 1}$ satisfying Equation (6).

- 可以将ODEs看作SDEs中 $\sigma_t = 0$ 的特殊情况
- SDE's numerical method from Euler-update to Euler-Maruyama method

$$X_{t+h} = X_t + hu_t(X_t) + \sqrt{h}\sigma_t\epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, I_d) \quad (9)$$

where $h = n^{-1} > 0$ is a step size hyperparameter for $n \in \mathbb{N}$. In other words, to simulate using the Euler-Maruyama method, we take a small step in the direction of $u_t(X_t)$ as well as add a little bit of Gaussian noise scaled by $\sqrt{h}\sigma_t$.

1.3.1.3 Diffusion Model's SDE

- $\frac{d}{dt}X_t = u_t^\theta(X_t) + \sigma_t dW_t$ ODE: 神经网络拟合的是vector field
- $X_0 = x, \quad x \sim p_{init} \quad (\Leftrightarrow X_0 \sim p_{init})$ SDE's initial conditions
- Remark: dW_t 即随机采样的Gaussian noisy作用，且 σ_t 为human fixed，不是网络学习的。

Algorithm 2 Sampling from a Diffusion Model (Euler-Maruyama method)

Require: Neural network u_t^θ , number of steps n , diffusion coefficient σ_t

- 1: Set $t = 0$
 - 2: Set step size $h = \frac{1}{n}$
 - 3: Draw a sample $X_0 \sim p_{init}$
 - 4: **for** $i = 1, \dots, n$ **do**
 - 5: Draw a sample $\epsilon \sim \mathcal{N}(0, I_d)$
 - 6: $X_{t+h} = X_t + hu_t^\theta(X_t) + \sigma_t\sqrt{h}\epsilon$
 - 7: Update $t \leftarrow t + h$
 - 8: **end for**
 - 9: **return** X_1
-

1.3.2 SDE(ODE) Generative Model Summary

Summary 7 (SDE generative model)

Throughout this document, a **diffusion model** consists of a neural network u_t^θ with parameters θ that parameterize a vector field and a fixed diffusion coefficient σ_t :

Neural network: $u^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d, (x, t) \mapsto u_t^\theta(x)$ with parameters θ

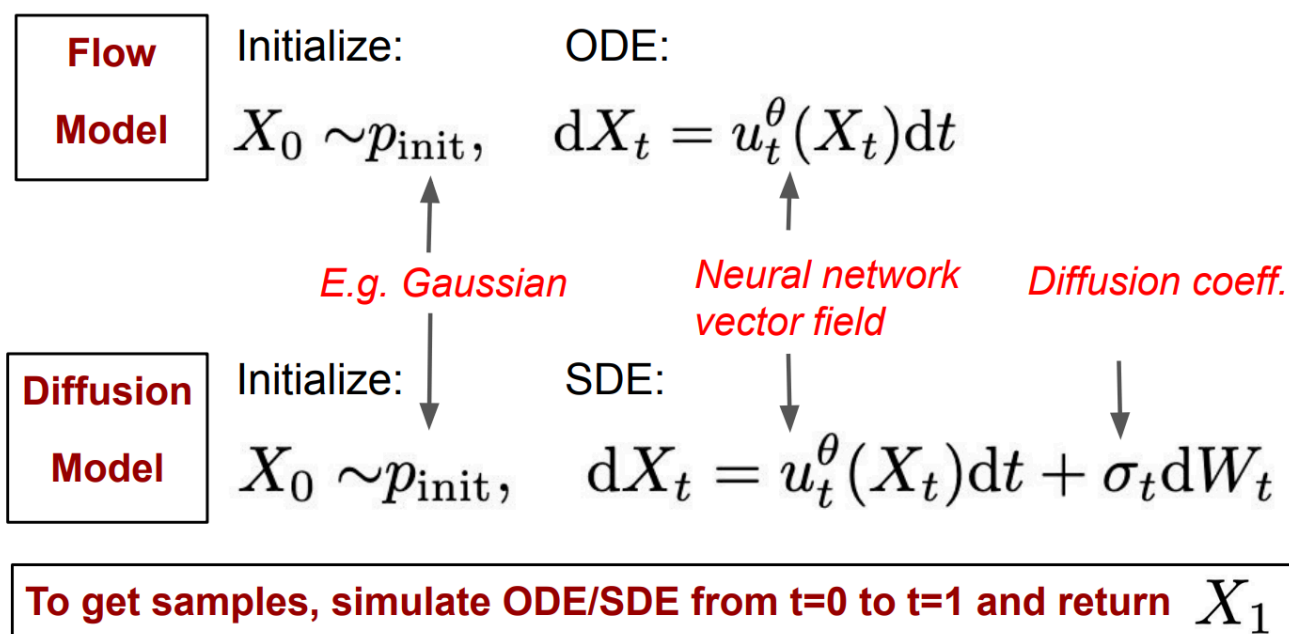
Fixed: $\sigma_t : [0, 1] \rightarrow [0, \infty), t \mapsto \sigma_t$

To obtain samples from our SDE model (i.e. generate objects), the procedure is as follows:

- | | |
|---|--|
| Initialization: $X_0 \sim p_{init}$ | ► Initialize with simple distribution, e.g. a Gaussian |
| Simulation: $dX_t = u_t^\theta(X_t)dt + \sigma_t dW_t$ | ► Simulate SDE from 0 to 1 |
| Goal: $X_1 \sim p_{data}$ | ► Goal is to make X_1 have distribution p_{data} |

A diffusion model with $\sigma_t = 0$ is a **flow model**.

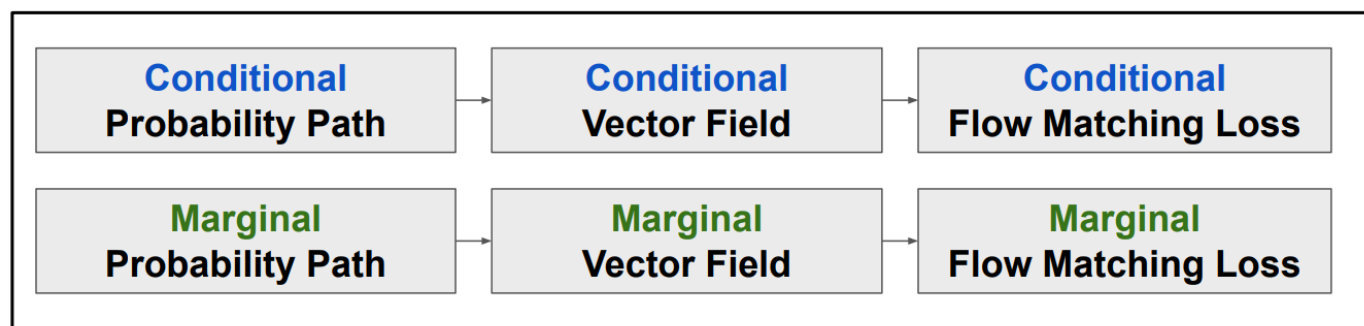
Reminder: Flow and Diffusion Models



1.4 Derive Flow Matching (Training A Flow Model)

本节的training algorithm围绕下面这张slide展开：

The Flow Matching Matrix



“Conditional” = “Per single data point”

“Marginal” = “Across distribution of data points”

1.4.1 Specify What a Probability Path



1. 根据上文关于“概率空间”中distribution transport, 即带有"概率密度"的空间点的flow, human design规定了 p_{init} , p_{data} 也已经存在。即ODE flow's trajectory需要满足 $X_0 \sim p_{init}$ for $t = 0$ and $X_1 \sim p_{data}$ for $t = 1$
2. 但是中间过程的轨迹依赖于vector field u 具有the freedom of design, 中间过程因为在 u 作用下, 所有"概率密度粒子"都在移动, t 时刻对应所有粒子的空间位置状态由flow描述($\psi_t(x)$, x is initial position), 此时将位于空间中同点的“粒子”的“概率密度”求和即得到该点的概率, 则可以认为fix t 的时候就对应一个distribution, 随着flow将 t 作为自变量, 那么flow就对应 $[0, 1]$ 的distribution transport, 被数学形式化称为probability path
3. 换言之, 不同的vector field u 与不同的probability path $X_t \sim p_t$ 是相互对应的。实践中, 我们可以设计的是probability path(just fullfil $p_0 = p_{init}$, $p_1 = p_{data}$), 然后让神经网络学习对应的vector field

1.4.2 Conditional / Marginal Probability Path

Remark: 这里的condition指的是已知data $z \sim p_{data} \in \mathcal{D}$, 而不是guidance condition (eg. Prompt)!

- 显然对于conditional probability path $p_t(\cdot|z)$ 的 $t = 1$ 时刻满足 $p_1(\cdot|z) = \delta_z$, for all $z \in \mathbb{R}^d$, 且可以证明 $p_0(\cdot|z) = p_{init}$ (可以通过 Eq(13)证明 $p_1(\cdot|z) = \delta_z$, $p_0(\cdot|z) = p_0 = p_{init} \Rightarrow p_0 = p_{init}$, $p_1 = p_{data}$ 满足我们的建模目标, 因而如此选择conditional probability path, 而非Note原文讲的marginal defined by conditonal)
- Marginal probability path can be calculated by conditional probability path

$$z \sim p_{data}, \quad x \sim p_t(\cdot|z) \Rightarrow x \sim p_t \quad \blacktriangleright \text{ sampling from marginal path} \quad (12)$$

$$p_t(x) = \int p_t(x|z)p_{data}(z)dz \quad \blacktriangleright \text{ density of marginal path} \quad (13)$$

$$p_0 = p_{init} \quad \text{and} \quad p_1 = p_{data}. \quad \blacktriangleright \text{ noise-data interpolation} \quad (14)$$

1.4.3 Gaussian Conditional Probability Path (common)

Example 8 (Gaussian Conditional Probability Path)

One particularly popular probability path is the **Gaussian probability path**. This is the **probability path used by most state-of-the-art models**. Let α_t, β_t be **noise schedulers**: two continuously differentiable, monotonic functions with $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1$. We then define the conditional probability path

$$p_t(\cdot|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d) \quad \blacktriangleright \text{ Gaussian conditional path} \quad (15)$$

which, by the conditions we imposed on α_t and β_t , fulfills

$$p_0(\cdot|z) = \mathcal{N}(\alpha_0 z, \beta_0^2 I_d) = \mathcal{N}(0, I_d), \quad \text{and} \quad p_1(\cdot|z) = \mathcal{N}(\alpha_1 z, \beta_1^2 I_d) = \delta_z,$$

where we have used the fact that a normal distribution with zero variance and mean z is just δ_z . Therefore, this choice of $p_t(x|z)$ fulfills Equation (11) for $p_{\text{init}} = \mathcal{N}(0, I_d)$ and is therefore a valid conditional interpolating path. In Figure 4, we illustrate its application to an image. We can express sampling from the marginal path p_t as:

$$z \sim p_{\text{data}}, \epsilon \sim p_{\text{init}} = \mathcal{N}(0, I_d) \Rightarrow x = \alpha_t z + \beta_t \epsilon \sim p_t \quad \blacktriangleright \text{ sampling from marginal Gaussian path} \quad (16)$$

Intuitively, the above procedure adds more noise for lower t until time $t = 0$, at which point there is only noise. In Figure 5, we plot an example of such an interpolating path.

Remark: Eq.(16) 参看 Eq.(12), 即 Eq.(16) 是 Eq.(12) 的实例化

1.4.4 Conditional / Marginal Vector Fields

- 如前1.4.2 所言, 不同的vector field u 与不同的probability path $X_t \sim p_t$ 是相互对应的, 我们希望找到对应的vector field u 满足设计的probability path。也将其分为conditional and marginal vector fields.
- Conditional vector field following:

For every data point $z \in \mathbb{R}^d$, let $u_t^{\text{target}}(\cdot|z)$ denote a **conditional vector field**. This can be any vector field such that corresponding ODE yields the conditional probability path $p_t(\cdot|z)$, i.e. such that it holds

$$X_0 \sim p_{\text{init}}, \quad \frac{d}{dt} X_t = u_t^{\text{target}}(X_t|z) \Rightarrow X_t \sim p_t(\cdot|z) \quad (0 \leq t \leq 1). \quad (17)$$

- Marginal vector field following:

Theorem 9 (Marginalization trick)

Let $u_t^{\text{target}}(x|z)$ be a conditional vector field (Equation (17)). Then the **marginal vector field** $u_t^{\text{target}}(x)$ defined as

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x|z) \frac{p_t(x|z)p_{\text{data}}(z)}{p_t(x)} dz, \quad (18)$$

follows the marginal probability path, i.e.

$$X_0 \sim p_{\text{init}}, \quad \frac{d}{dt} X_t = u_t^{\text{target}}(X_t) \Rightarrow X_t \sim p_t \quad (0 \leq t \leq 1). \quad (19)$$

In particular, $X_1 \sim p_{\text{data}}$ for this ODE, so that we might say " u_t^{target} converts noise p_{init} into data p_{data} ".

Remark: $u_t^{\text{target}}(x) = u_t^{\text{target}}(x|z)$ 从 x 向 z 速度向量的加权, 其中加权权重为posterior $p(z|x)$

1.4.5 Conditional Vector Field of Conditional Gaussian Probability Path (common)

- 承接1.4.3 common conditional Gaussian probability path导出其对应的conditional vector field

Example 10 (Target ODE for Gaussian probability paths)

As before, let $p_t(\cdot|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$ for noise schedulers α_t, β_t (see Equation (15)). Let $\dot{\alpha}_t = \partial_t \alpha_t$ and $\dot{\beta}_t = \partial_t \beta_t$ denote respective time derivatives of α_t and β_t . Here, we want to show that the **conditional Gaussian vector field** given by

$$u_t^{\text{target}}(x|z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x \quad (20)$$

is a valid conditional vector field model in the sense of Theorem 9: its ODE trajectories X_t satisfy $X_t \sim p_t(\cdot|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$ if $X_0 \sim \mathcal{N}(0, I_d)$. In Figure 6, we confirm this visually by comparing samples from the conditional probability path (ground truth) to samples from simulated ODE trajectories of this flow. As you can see, the distribution match. We will now prove this.

Proof. Let us construct a conditional flow model $\psi_t^{\text{target}}(x|z)$ first by defining

$$\psi_t^{\text{target}}(x|z) = \alpha_t z + \beta_t x. \quad (21)$$

If X_t is the ODE trajectory of $\psi_t^{\text{target}}(\cdot|z)$ with $X_0 \sim p_{\text{init}} = \mathcal{N}(0, I_d)$, then by definition

$$X_t = \psi_t^{\text{target}}(X_0|z) = \alpha_t z + \beta_t X_0 \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d) = p_t(\cdot|z).$$

We conclude that the trajectories are distributed like the conditional probability path (i.e, Equation (17) is fulfilled). It remains to extract the vector field $u_t^{\text{target}}(x|z)$ from $\psi_t^{\text{target}}(x|z)$. By the definition of a flow (Equation (2b)), it holds

$$\begin{aligned} \frac{d}{dt} \psi_t^{\text{target}}(x|z) &= u_t^{\text{target}}(\psi_t^{\text{target}}(x|z)|z) \quad \text{for all } x, z \in \mathbb{R}^d \\ \stackrel{(i)}{\Leftrightarrow} \quad \dot{\alpha}_t z + \dot{\beta}_t x &= u_t^{\text{target}}(\alpha_t z + \beta_t x|z) \quad \text{for all } x, z \in \mathbb{R}^d \\ \stackrel{(ii)}{\Leftrightarrow} \quad \dot{\alpha}_t z + \dot{\beta}_t \left(\frac{x - \alpha_t z}{\beta_t} \right) &= u_t^{\text{target}}(x|z) \quad \text{for all } x, z \in \mathbb{R}^d \\ \stackrel{(iii)}{\Leftrightarrow} \quad \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x &= u_t^{\text{target}}(x|z) \quad \text{for all } x, z \in \mathbb{R}^d \end{aligned}$$

where in (i) we used the definition of $\psi_t^{\text{target}}(x|z)$ (Equation (21)), in (ii) we reparameterized $x \rightarrow (x - \alpha_t z)/\beta_t$, and in (iii) we just did some algebra. Note that the last equation is the conditional Gaussian vector field as we defined in Equation (20). This proves the statement.^a $\square$

 Synced Block [Export is restricted for this type of content.]

1.4.6 Continuity Equation

- Remark: 即某点“概率密度”的变化率+该点速度向量场该点的“概率密度”的散度(流量)=0 (守恒观点)

Theorem 11 (Continuity Equation)

Let us consider an flow model with vector field u_t^{target} with $X_0 \sim p_{\text{init}} = p_0$. Then $X_t \sim p_t$ for all $0 \leq t \leq 1$ if and only if

$$\partial_t p_t(x) = -\text{div}(p_t u_t^{\text{target}})(x) \quad \text{for all } x \in \mathbb{R}^d, 0 \leq t \leq 1, \quad (23)$$

where $\partial_t p_t(x) = \frac{d}{dt} p_t(x)$ denotes the time-derivative of $p_t(x)$. Equation 23 is known as the **continuity equation**.

- “守恒”体现在：对整个固定体积 V 积分，并利用高斯散度定理（换言之可以用高斯散度定理

$$\text{证明}) : \frac{d}{dt} \int_V \rho dV = - \int_V \nabla \cdot (\rho \mathbf{v}) dV = - \oint_{\partial V} \rho \mathbf{v} \cdot d\mathbf{S}$$

左边是体积 V 内总质量的变化率，右边是通过边界曲面流出的总质量流。**总质量变化完全由通过边界的净流出决定**，这就是全局质量守恒。（这里的质量可以等效替换为概率，密度替换为概率密度）

- Remark：可以用连续方程证明marginal vector field = conditional vector field 加权 posterior

Proof of Theorem 9. By Theorem 11, we have to show that the marginal vector field u_t^{target} , as defined as in Equation (18), satisfies the continuity equation. We can do this by direct calculation:

$$\begin{aligned} \partial_t p_t(x) &\stackrel{(i)}{=} \partial_t \int p_t(x|z) p_{\text{data}}(z) dz = \int \partial_t p_t(x|z) p_{\text{data}}(z) dz \\ &\stackrel{(ii)}{=} \int -\text{div}(p_t(\cdot|z) u_t^{\text{target}}(\cdot|z))(x) p_{\text{data}}(z) dz \\ &\stackrel{(iii)}{=} -\text{div} \left(\int p_t(x|z) u_t^{\text{target}}(x|z) p_{\text{data}}(z) dz \right) \\ &\stackrel{(iv)}{=} -\text{div} \left(p_t(x) \int u_t^{\text{target}}(x|z) \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} dz \right) (x) \\ &\stackrel{(v)}{=} -\text{div} (p_t u_t^{\text{target}})(x), \end{aligned}$$

where in (i) we used the definition of $p_t(x)$ in Equation (12), in (ii) we used the continuity equation for the conditional probability path $p_t(\cdot|z)$, in (iii) we swapped the integral and divergence operator using Equation (22), in (iv) we multiplied and divided by $p_t(x)$, and in (v) we used Equation (18). The beginning and end of the above chain of equations show that the continuity equation is fulfilled for u_t^{target} . By Theorem 11, this is enough to imply Equation (19), and we are done. $\square$

1.4.7 Learning the Marginal Vector Field

- 由上可知，我们将建模 p_{data} 的问题转化为让神经网络学习“从 p_{init} 到 p_{data} ”的 marginal vector field u_t^{target} ，因此我们的目标是 u_t^θ 去近似拟合 u_t^{target} ，显然会想到用 mean-squared error。flow matching loss定义如下：

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, x \sim p_t} [\|u_t^\theta(x) - u_t^{\text{target}}(x)\|^2] \quad (24)$$

$$\stackrel{(i)}{=} \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x) - u_t^{\text{target}}(x)\|^2], \quad (25)$$

Remark: 问题在于对于 p_{init} 和 p_{data} 我们只知道transport's start distribution, end distribution不可知，难以设计marginal probability path，则flow不可得，那么 $u_t^{\text{target}}(\cdot)$ 即为未知，故flow matching loss中的监督标签 $u_t^{\text{target}}(x)$ 不可得，无法训练

- 联想上文的given data z , conditional probability path的 $p_0 = p_{init}$, $p_1 = \delta_z$, 且已设计了 common conditional Gaussian probability path $p_t(\cdot|z) \sim N(\alpha_t z, \beta_t I)$, where $\alpha_0 = \beta_1 = 0$, $\alpha_1 = \beta_0 = 1$, 且得到了相应的conditional vector field $u_t^{\text{target}}(x|z)$, 即可以类似的定义 conditional flow matching loss:

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2]. \quad (26)$$

 Synced Block [Export is restricted for this type of content.]

- $\mathcal{L}_{FM} = \mathcal{L}_{CFM} + C$ 的证明过程following

Direct Proof. The proof works by expanding the mean-squared error into three components and removing constants:

$$\begin{aligned} \mathcal{L}_{FM}(\theta) &\stackrel{(i)}{=} \mathbb{E}_{t \sim \text{Unif}, x \sim p_t} [\|u_t^\theta(x) - u_t^{\text{target}}(x)\|^2] \\ &\stackrel{(ii)}{=} \mathbb{E}_{t \sim \text{Unif}, x \sim p_t} [\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\text{target}}(x) + \|u_t^{\text{target}}(x)\|^2] \\ &\stackrel{(iii)}{=} \mathbb{E}_{t \sim \text{Unif}, x \sim p_t} [\|u_t^\theta(x)\|^2] - 2\mathbb{E}_{t \sim \text{Unif}, x \sim p_t} [u_t^\theta(x)^T u_t^{\text{target}}(x)] + \underbrace{\mathbb{E}_{t \sim \text{Unif}_{[0,1]}, x \sim p_t} [\|u_t^{\text{target}}(x)\|^2]}_{=: C_1} \\ &\stackrel{(iv)}{=} \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x)\|^2] - 2\mathbb{E}_{t \sim \text{Unif}, x \sim p_t} [u_t^\theta(x)^T u_t^{\text{target}}(x)] + C_1 \end{aligned}$$

where (i) holds by definition, in (ii) we used the formula $\|a - b\|^2 = \|a\|^2 - 2a^T b + \|b\|^2$, in (iii) we define a constant C_1 and in (iv) we used the sampling procedure of p_t given by Equation (12). Let us reexpress the second summand:

$$\begin{aligned} \mathbb{E}_{t \sim \text{Unif}, x \sim p_t} [u_t^\theta(x)^T u_t^{\text{target}}(x)] &\stackrel{(i)}{=} \int_0^1 \int p_t(x) u_t^\theta(x)^T u_t^{\text{target}}(x) dx dt \\ &\stackrel{(ii)}{=} \int_0^1 \int p_t(x) u_t^\theta(x)^T \left[\int u_t^{\text{target}}(x|z) \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} dz \right] dx dt \\ &\stackrel{(iii)}{=} \int_0^1 \int \int u_t^\theta(x)^T u_t^{\text{target}}(x|z) p_t(x|z) p_{\text{data}}(z) dz dx dt \\ &\stackrel{(iv)}{=} \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} [u_t^\theta(x)^T u_t^{\text{target}}(x|z)] \end{aligned}$$

where in (i) we expressed the expected value as an integral, in (ii) we use Equation (18), in (iii) we use the fact that integrals are linear, in (iv) we express the integral as an expected value. Note that this was really the crucial step of the proof: The beginning of the equality used the marginal vector field $u_t^{\text{target}}(x)$, while the end uses the conditional vector field $u_t^{\text{target}}(x|z)$. We plug is into the equation for $\mathcal{L}_{FM}$ to get:

$$\begin{aligned} \mathcal{L}_{FM}(\theta) &\stackrel{(i)}{=} \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x)\|^2] - 2\mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} [u_t^\theta(x)^T u_t^{\text{target}}(x|z)] + C_1 \\ &\stackrel{(ii)}{=} \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\text{target}}(x|z) + \|u_t^{\text{target}}(x|z)\|^2 - \|u_t^{\text{target}}(x|z)\|^2] + C_1 \\ &\stackrel{(iii)}{=} \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} [\|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2] + \underbrace{\mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} [-\|u_t^{\text{target}}(x|z)\|^2]}_{C_2} + C_1 \\ &\stackrel{(iv)}{=} \mathcal{L}_{CFM}(\theta) + \underbrace{C_2 + C_1}_{=: C} \end{aligned}$$

where in (i) we plugged in the derived equation, in (ii) we added and subtracted the same value, in (iii) we used the formula $\|a - b\|^2 = \|a\|^2 - 2a^T b + \|b\|^2$ again, and in (iv) we defined a constant in θ . This finishes the proof. $\square$

- 训练的优势：训练过程无需模拟ODE的计算过程，可以”一步到位“监督简单的regression object (mean square error)
- 将common Gaussian probability path's vector field代入，进一步再human design α_t, β_t 的形式(下例为线性)

Example 13 (Flow Matching for Gaussian Conditional Probability Paths)

Let us return to the example of Gaussian probability paths $p_t(\cdot|z) = \mathcal{N}(\alpha_t z; \beta_t^2 I_d)$, where we may sample from the conditional path via

$$\epsilon \sim \mathcal{N}(0, I_d) \Rightarrow x_t = \alpha_t z + \beta_t \epsilon \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d) = p_t(\cdot|z). \quad (28)$$

As we derived in Equation (20), the conditional vector field $u_t^{\text{target}}(x|z)$ is given by

$$u_t^{\text{target}}(x|z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x, \quad (29)$$

where $\dot{\alpha}_t = \partial_t \alpha_t$ and $\dot{\beta}_t = \partial_t \beta_t$ are the respective time derivatives. Plugging in this formula, the conditional flow matching loss reads

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d)} [\|u_t^\theta(x) - \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z - \frac{\dot{\beta}_t}{\beta_t} x\|^2] \quad (30)$$

$$\stackrel{(i)}{=} \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, \epsilon \sim \mathcal{N}(0, I_d)} [\|u_t^\theta(\alpha_t z + \beta_t \epsilon) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon)\|^2] \quad (31)$$

where in (i) we plugged in Equation (28) and replaced x by $\alpha_t z + \beta_t \epsilon$. Note the simplicity of $\mathcal{L}_{\text{CFM}}$: We sample a data point z , sample some noise ϵ and then we take a mean squared error. Let us make this even more concrete for the special case of $\alpha_t = t$, and $\beta_t = 1 - t$. The corresponding probability $p_t(x|z) = \mathcal{N}(tz, (1-t)^2)$ is sometimes referred to as the (Gaussian) **CondOT probability path**. Then we have $\dot{\alpha}_t = 1, \dot{\beta}_t = -1$, so that

$$\mathcal{L}_{\text{cfm}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, \epsilon \sim \mathcal{N}(0, I_d)} [\|u_t^\theta(tz + (1-t)\epsilon) - (z - \epsilon)\|^2]$$

Common Training Algorithm伪代码

Algorithm 3 Flow Matching Training Procedure (for Gaussian CondOT path $p_t(x|z) = \mathcal{N}(tz, (1-t)^2)$)

Require: A dataset of samples $z \sim p_{\text{data}}$, neural network u_t^θ

1: **for** each mini-batch of data **do**

2: Sample a data example z from the dataset.

3: Sample a random time $t \sim \text{Unif}_{[0,1]}$.

4: Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$

5: Set

$$x = tz + (1-t)\epsilon \quad (\text{General case: } x \sim p_t(\cdot|z))$$

6: Compute loss

$$\mathcal{L}(\theta) = \|u_t^\theta(x) - (z - \epsilon)\|^2 \quad (\text{General case: } = \|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2)$$

7: Update $\theta \leftarrow \text{grad_update}(\mathcal{L}(\theta))$.

8: **end for**

1.4.8 Why Euler-Update Many Times

- 在笔者阅读论文中，常见将flow matching中的conditional Gaussian probability path $p(\cdot|z) \sim \mathcal{N}((tz, (1-t)I)$ 称作最优传输路径，直观上理解就是一条“匀速直线轨迹”。因而笔者在未深入了解CFM与FM前，常困惑于既然最优传输路径为human designed line，在采样生成时的simulate euler-update为什么不能一步更新到位？（因为那时候并没有理解到这其实是CFM's conditional probability path）
- Human designed optimistic transport path是conditional probability path，对应 conditional vector field是“匀速直线”，我们利用 $\mathcal{L}_{\text{CFM}}$ 等效 $\mathcal{L}_{\text{FM}}$ 训练拟合marginal vector field，但此时其并非一定是“匀速直线”！【TODO  用户191488 可以通过数值模拟举例来观察】，所以需要多步的euler-update来保证传输的正确性。

【TODO  用户191488 Conditional Probability Path与Marginal Probability Path的“几何观感”的关系，eg. Conditional Probability Path走line，Marginal Probability Path会怎么样？】

1.5 Score Function and Score Matching

1.5.1 What is the "Score Function"

- 在flow matching中，感兴趣的是学习vector field $u_t(x)$ ；在diffusion早于flow matching的machine learning中是用另一种视角"score function" $\nabla_x \log q(x)$, the gradient of the log-likelihood of q with respect to x 来进行学习的。
- 换言之，Flow matching中的vector field $u_t(x)$ 、 $u_t(x|z)$ 与Score Matching中的score function $\nabla_x \log p_t(x)$ 、 $\nabla_x \log p_t(x|z)$ 是等价的reparameter replace。详细可见 [1.5.4](#)。

1.5.2 Conditional and Maginal Score Funtion

- 类似 conditional vector field 与 marginal vector field的关系有：

Hence, **the relation between the conditional and marginal score is analogous to the relation between the conditional and marginal vector field**. Note that we can prove [Equation \(38\)](#) via

$$\nabla \log p_t(x) = \frac{\nabla p_t(x)}{p_t(x)} = \frac{\nabla \int p_t(x|z)p_{\text{data}}(z)dz}{p_t(x)} = \frac{\int \nabla p_t(x|z)p_{\text{data}}(z)dz}{p_t(x)} = \int \nabla \log p_t(x|z) \frac{p_t(x|z)p_{\text{data}}(z)}{p_t(x)} dz, \quad (39)$$

1.5.3 Score Function for Gaussian Probability Path (Common)

Example 15 (Score Function for Gaussian Probability Paths.)

For the Gaussian path $p_t(x|z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$, we can use the form of the Gaussian probability density (see [Equation \(97\)](#)) to get

$$\nabla \log p_t(x|z) = \nabla \log \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d) = -\frac{x - \alpha_t z}{\beta_t^2}. \quad (40)$$

Remark: 可以发现Common Gaussian Probability Path中，conditional score funtion是 x 、 z 的线性组合，而 [1.4](#) 中的conditional vecor field 也是 x 、 z 的线性组合。那么二者是可以reparameter做替换的。

1.5.4 Conversion Formula of Score Function and Vector Field For Gaussian Probailty Path

Proposition 1 (Conversion Formula for Gaussian Probability Paths)

For the Gaussian probability path $p_t(x|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$, the conditional (resp. marginal) vector field and the conditional (resp. marginal) score are related by the following identities

$$u_t^{\text{target}}(x|z) = a_t \nabla \log p_t(x|z) + b_t x, \quad a_t = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right), \quad b_t = \frac{\dot{\alpha}_t}{\alpha_t} \quad (41)$$

$$u_t^{\text{target}}(x) = a_t \nabla \log p_t(x) + b_t x. \quad (42)$$

In particular, we note that the conditional (resp. marginal) vector field can be recovered from the conditional (resp. marginal) score, and vice versa.

◦ 证明following:

Proof. For the conditional vector field and conditional score, we can derive:

$$u_t^{\text{target}}(x|z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x \stackrel{(i)}{=} \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \left(\frac{\alpha_t z - x}{\beta_t^2} \right) + \frac{\dot{\alpha}_t}{\alpha_t} x = \left(\beta_t^2 \frac{\dot{\alpha}_t}{\alpha_t} - \dot{\beta}_t \beta_t \right) \nabla \log p_t(x|z) + \frac{\dot{\alpha}_t}{\alpha_t} x$$

where in (i) we just did some algebra. By taking integrals, the same identity holds for the marginal flow vector field and the marginal score function:

$$\begin{aligned} u_t^{\text{target}}(x) &= \int u_t^{\text{target}}(x|z) \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} dz = \int [a_t \nabla \log p_t(x|z) + b_t x] \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} dz \\ &\stackrel{(i)}{=} a_t \nabla \log p_t(x) + b_t x \end{aligned}$$

where in (i) we used Equation (38) and the fact that posterior density integrates to 1. □

◦ Remark:

- 由上变换可知，学习vector field也就是学习score funtion，学习score funtion也就是学习vector field。只要神经网络学会一个，理论上都可以通过代数计算得到另一个
- 因而在早期的diffusion工作中，都是以score funtion的视角来训练神经网络学习 $\nabla \log p_t(x)$

1.5.5 SDE's Probability Path

- 对于一般的“概率密度粒子”在空间SDE的distribution transport，可以求的对应的probability path必定满足Fokker-Planck Equation。（证明详见Lecture Note Section B）

Theorem 19 (Fokker-Planck Equation)

Let p_t be a probability path and let us consider the SDE

$$X_0 \sim p_{\text{init}}, \quad dX_t = u_t(X_t)dt + \sigma_t dW_t.$$

Then X_t has distribution p_t for all $0 \leq t \leq 1$ if and only if the **Fokker-Planck equation** holds:

$$\partial_t p_t(x) = -\text{div}(p_t u_t)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \quad \text{for all } x \in \mathbb{R}^d, 0 \leq t \leq 1, \quad (49)$$

1.5.6 Relation between SDE and ODE

- 在上文ODE's vector field u_t^{target} 映射one solution flow也就对应one probability path。在SDE中视作ODE+Brown motion (noisy drift)。可以证明只需任何一个ODE都有一个SDE与之等效，即会产生相同的probability path
- 物理直觉，Brown noisy drift stochastic dynamic 需要添加一个的引导修正项，即 $\frac{\sigma_t^2}{2} \nabla \log p_t(X_t)$ ，才能与ODE等价的probability path

$$X_0 \sim p_{init}, \quad dX_t = u_t^{target}(X_t)dt + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t)dt + \sigma_t dW_t \quad (44)$$

$$= \left[u_t^{target}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right] dt + \sigma_t dW_t$$

$$\Rightarrow X_t \sim p_t \quad (0 \leq t \leq 1). \quad (45)$$

where $X_t \sim p_t$ is ODE: $dX_t = u_t^{target}(X_t)dt$, $X_0 \sim p_{init}$ 对应的probability path

- 通过 1.5.5 引理可证following:
 - 先定义Laplacian operator Δ :

$$\Delta w_t(x) = \sum_{i=1}^d \frac{\partial^2}{\partial x_i^2} w_t(x) = \text{div}(\nabla w_t)(x), \quad (48)$$

for scalar field $w_t : \mathbb{R}^d \rightarrow \mathbb{R}$.

Proof of Theorem 17. By Theorem 19, we need to show that that the SDE defined in Equation (44) satisfies the Fokker-Planck equation for p_t . We can do this by direction calculation:

$$\begin{aligned} \partial_t p_t(x) &\stackrel{(i)}{=} -\text{div}(p_t u_t^{target})(x) \\ &\stackrel{(ii)}{=} -\text{div}(p_t u_t^{target})(x) - \frac{\sigma_t^2}{2} \Delta p_t(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \\ &\stackrel{(iii)}{=} -\text{div}(p_t u_t^{target})(x) - \text{div}\left(\frac{\sigma_t^2}{2} \nabla p_t\right)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \\ &\stackrel{(iv)}{=} -\text{div}(p_t u_t^{target})(x) - \text{div}\left(p_t \left[\frac{\sigma_t^2}{2} \nabla \log p_t\right]\right)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x) \\ &\stackrel{(v)}{=} -\text{div}\left(p_t \left[u_t^{target} + \frac{\sigma_t^2}{2} \nabla \log p_t\right]\right)(x) + \frac{\sigma_t^2}{2} \Delta p_t(x), \end{aligned}$$

where in (i) we used Theorem 11, in (ii) we added and subtracted the same term, in (iii) we used the definition of the Laplacian (Equation (48)), in (iv) we used that $\nabla \log p_t = \frac{\nabla p_t}{p_t}$, and in (v) we used the linearity of the divergence operator. The above derivation shows that the SDE defined in Equation (44) satisfies the Fokker-Planck equation for p_t . By Theorem 19, this implies $X_t \sim p_t$ for $0 \leq t \leq 1$, as desired. $\square$

- 将其实例化到common Gaussian Probability Path，即用score funtion代入vector field u :

Example 18 (Gaussian SDE Extension Trick)

By Proposition 1, for Gaussian probability paths, we can express the SDE from Theorem 17 purely using score functions:

$$X_0 \sim p_{init}, \quad dX_t = \left[\left(a_t + \frac{\sigma_t^2}{2} \right) \nabla \log p_t(X_t) + b_t X_t \right] dt + \sigma_t dW_t \quad (46)$$

$$\Rightarrow X_t \sim p_t \quad (0 \leq t \leq 1) \quad (47)$$

where a_t, b_t are defined as in Proposition 1.

1.5.7 Score Matching

- 已经证明了score function是vector field的reparameter替换，可以认为二者是等效的，类比flow matching直接得到score matching:

we can also learn marginal score functions directly. To approximate the marginal score $\nabla \log p_t$, we use a neural network that we call **score network** $s_t^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$. In the same way as before, we can design a **score matching** loss and a **denoising score matching** loss:

$$\begin{aligned}\mathcal{L}_{\text{SM}}(\theta) &= \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} \left[\|s_t^\theta(x) - \nabla \log p_t(x)\|^2 \right] &> \text{score matching loss} \\ \mathcal{L}_{\text{CSM}}(\theta) &= \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} \left[\|s_t^\theta(x) - \nabla \log p_t(x|z)\|^2 \right] &> \text{conditional score matching loss}\end{aligned}$$

where again the difference is using the marginal score $\nabla \log p_t(x)$ vs. using the conditional score $\nabla \log p_t(x|z)$. As before, we ideally would want to minimize the score matching loss but can't because we don't know $\nabla \log p_t(x)$. But similarly as before, the denoising score matching loss is a tractable alternative:

Theorem 22

The score matching loss equals the denoising score matching loss up to a constant:

$$\mathcal{L}_{\text{SM}}(\theta) = \mathcal{L}_{\text{CSM}}(\theta) + C,$$

where C is independent of parameters θ . Therefore, their gradients coincide:

$$\nabla_\theta \mathcal{L}_{\text{SM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{CSM}}(\theta).$$

In particular, for the minimizer θ^* , it will hold that $s_t^{\theta^*} = \nabla \log p_t$.

Proof. Note that the formula for $\nabla \log p_t$ (Equation (38)) looks the same as the formula for u_t^{target} (Equation (18)). Therefore, the proof is identical to the proof of Theorem 12 replacing u_t^{target} with $\nabla \log p_t$. $\square$

- 进一步将Loss作 human-designed common conditional Gaussian Probability Path的实例化:

Example 23 (Denoising Diffusion Models: Score Matching for Gaussian Probability Paths)

Let us instantiate the denoising score matching loss for the case of $p_t(x|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$. As we derived in Equation (40), the conditional score $\nabla \log p_t(x|z)$ has the formula

$$\nabla \log p_t(x|z) = -\frac{x - \alpha_t z}{\beta_t^2}. \quad (51)$$

Plugging in this formula, the conditional score matching loss becomes:

$$\begin{aligned}\mathcal{L}_{\text{CSM}}(\theta) &= \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} \left[\left\| s_t^\theta(x) + \frac{x - \alpha_t z}{\beta_t^2} \right\|^2 \right] \\ &\stackrel{(i)}{=} \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, \epsilon \sim \mathcal{N}(0, I_d)} \left[\left\| s_t^\theta(\alpha_t z + \beta_t \epsilon) + \frac{\epsilon}{\beta_t} \right\|^2 \right] \\ &= \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, \epsilon \sim \mathcal{N}(0, I_d)} \left[\frac{1}{\beta_t^2} \left\| \beta_t s_t^\theta(\alpha_t z + \beta_t \epsilon) + \epsilon \right\|^2 \right]\end{aligned}$$

where in (i) we plugged in Equation (28) and replaced x by $\alpha_t z + \beta_t \epsilon$. Note that the network s_t^θ essentially learns to predict the noise that was used to corrupt a data sample z . This explains why the above training loss is called **denoising score matching**. It was soon realized that the above loss is numerically unstable for $\beta_t \approx 0$ close to zero (i.e. denoising score matching only works if you add a sufficient amount of noise). In some of the first works on denoising diffusion models (see **Denoising Diffusion Probabilistic Models**, [17]) it was therefore proposed to drop the constant $\frac{1}{\beta_t^2}$ in the loss and reparameterize s_t^θ into a **noise predictor** network $\epsilon_t^\theta : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ via:

$$-\beta_t s_t^\theta(x) = \epsilon_t^\theta(x) \quad \Rightarrow \quad \mathcal{L}_{\text{DDPM}}(\theta) = \mathbb{E}_{t \sim \text{Unif}, z \sim p_{\text{data}}, \epsilon \sim \mathcal{N}(0, I_d)} \left[\left\| \epsilon_t^\theta(\alpha_t z + \beta_t \epsilon) - \epsilon \right\|^2 \right]$$

As before, the network ϵ_t^θ essentially learns to predict the noise that was used to corrupt a data sample z . In Algorithm 4, we summarize the training procedure.

- DDPM对应的伪代码如下：

Algorithm 4 Score Matching Training Procedure for Gaussian probability path

Require: A dataset of samples $z \sim p_{\text{data}}$, score network s_t^θ or noise predictor ϵ_t^θ

- 1: **for** each mini-batch of data **do**
- 2: Sample a data example z from the dataset.
- 3: Sample a random time $t \sim \text{Unif}_{[0,1]}$.
- 4: Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
- 5: Set $x_t = \alpha_t z + \beta_t \epsilon$ (General case: $x_t \sim p_t(\cdot|z)$)
- 6: Compute loss

$$\mathcal{L}(\theta) = \left\| s_t^\theta(x_t) + \frac{\epsilon}{\beta_t} \right\|^2 \quad \text{(General case: } = \|s_t^\theta(x_t) - \nabla \log p_t(x_t|z)\|^2)$$

$$\text{Alternatively: } \mathcal{L}(\theta) = \left\| \epsilon_t^\theta(x_t) - \epsilon \right\|^2$$

- 7: Update the model parameters θ via gradient descent on $\mathcal{L}(\theta)$.
 - 8: **end for**
-

1.6 Guidance Flow Matching: Conditioned on a Prompt

之前讨论的都是none-guidance generation，即建模 $p_{\text{data}}(z)$ ，但在实际应用中希望generate an object conditioned on some additional information y ，比如根据prompt生成基于prompt的图片，而不是生成一张合理的随机照片，即希望从 $p_{\text{data}}(z|y)$ 中采样生成，而不是从 $p_{\text{data}}(z)$ 中采样生成。为了避免和上文的给定 z 的conditional相混淆，这里全部对 y 的相应描述都使用guidance，望读者区分。

1.6.1 朴素改良

- 类比none-guidence situation，只需要让 u_t^θ 学习不同guidence y 下将Gaussian Distribution p_{init} transport to guideded data distribution $p_{data}(z|y)$ ，根据FM-CFM的结论就是要最小化following:

$$\mathbb{E}_{z \sim p_{data}(\cdot|y), x \sim p_t(\cdot|z)} \|u_t^\theta(x|y) - u_t^{\text{target}}(x|z)\|^2. \quad (57)$$

- 因为希望神经网络可以根据不同guidence y 生成object，那么就需要神经网络学习根据不同的 y 输出不同 $p_{init} \rightarrow p_{data}(z|y)$ 的vector field $u_t^\theta(x|y)$ ，因此在FM基础上将guidence y 也作为输入。对应的data distribution就应该是 $p_{data}(z, y)$ ，训练的时候从中采样训练 $\mathcal{L}_{CFM}^{\text{guided}}(\theta)$

$$\mathcal{L}_{CFM}^{\text{guided}}(\theta) = \mathbb{E}_{(z,y) \sim p_{data}(z,y), t \sim \text{Unif}[0,1], x \sim p_t(\cdot|z)} \|u_t^\theta(x|y) - u_t^{\text{target}}(x|z)\|^2. \quad (58)$$

- 推理时与原来的Euler update相同，因为guidence y 输入会确定对应的vector field。（可以理解为将原来的神经网络拟合一个向量场改良为神经网络拟合关于guidence y 到向量场函数的映射(非单个向量场)）

We define a **guided diffusion model** to consist of a **guided vector field** $u_t^\theta(\cdot|y)$, parameterized by some neural network, and a time-dependent diffusion coefficient σ_t , together given by

$$\text{Neural network: } u^\theta : \mathbb{R}^d \times \mathcal{Y} \times [0, 1] \rightarrow \mathbb{R}^d, (x, y, t) \mapsto u_t^\theta(x|y)$$

$$\text{Fixed: } \sigma_t : [0, 1] \rightarrow [0, \infty), t \mapsto \sigma_t$$

Notice the difference from summary 7: we are additionally guiding u_t^θ with the input $y \in \mathcal{Y}$. For any such $y \in \mathcal{Y}$, samples may then be generated from such a model as follows:

Initialization: $X_0 \sim p_{init}$	► Initialize with simple distribution (such as a Gaussian)
Simulation: $dX_t = u_t^\theta(X_t y)dt + \sigma_t dW_t$	► Simulate SDE from $t = 0$ to $t = 1$.
Goal: $X_1 \sim p_{data}(\cdot y)$	► Goal is for X_1 to be distributed like $p_{data}(\cdot y)$.

When $\sigma_t = 0$, we say that such a model is a **guided flow model**. In the following, we restrict ourselves to flow

但是该方法会出现guidence的条件引导性不够，表现在文生图中生成的图片对于文字的契合度不够高，因而导出下面想要增强guidence的research idea

1.6.2 Classifier Guidance

- 根据上文vector field与score function的关系，可以对 $u_t^{\text{target}}(x|y)$ 进行代数变形：

$$u_t^{\text{target}}(x|y) = a_t \nabla \log p_t(x|y) + b_t x, \quad (59)$$

Next, realize that $p_t(x|y)$ is a conditional density. Hence, we can use Bayes' rule to rewrite the guided score as

$$p_t(x|y) = \frac{p_t(x)p_t(y|x)}{p_t(y)} \quad (60)$$

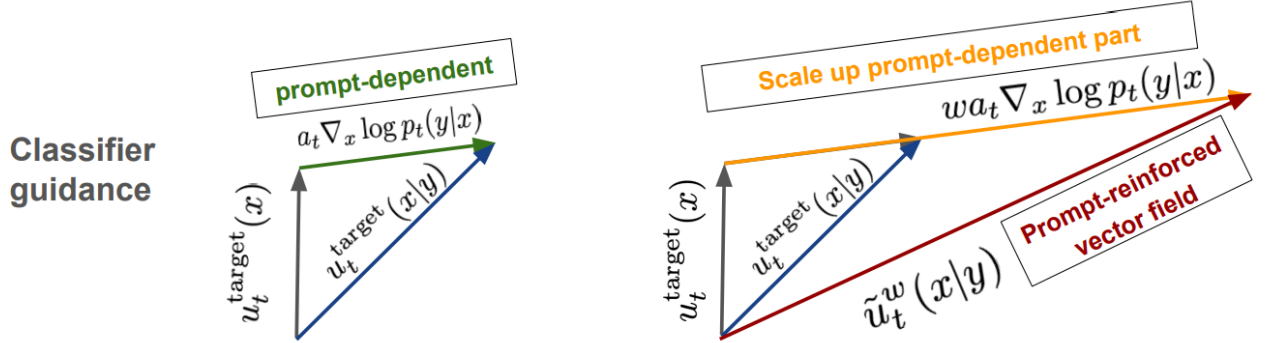
$$\nabla \log p_t(x|y) = \nabla \log \left(\frac{p_t(x)p_t(y|x)}{p_t(y)} \right) = \nabla \log p_t(x) + \nabla \log p_t(y|x), \quad (61)$$

where we used that the gradient ∇ is taken with respect to the variable x , so that $\nabla \log p_t(y) = 0$. We may thus rewrite

$$u_t^{\text{target}}(x|y) = b_t x + a_t (\nabla \log p_t(x) + \nabla \log p_t(y|x)) = u_t^{\text{target}}(x) + a_t \nabla \log p_t(y|x).$$

- 观察到 $u_t^{\text{target}}(x|y)$ 可以分为 $u_t^{\text{target}}(x)$ 与 $\nabla \log p_t(y|x)$ 。Assumption: $u_t^{\text{target}}(x)$ 保证生成的合理性， $\nabla \log p_t(y|x)$ 对应prompt-dependent。因而直观的想法是认为增加 $\nabla \log p_t(y|x)$ 的比例

$$\tilde{u}_t(x|y) = u_t^{\text{target}}(x) + w a_t \nabla \log p_t(y|x), \quad (\text{classifier guidance}) \quad (62)$$



Remark:

- where $w > 1$ 被称作 guidance scale

这里的 $\nabla_x \log p_t(y|x)$ 对应一个classifier分类器，需要采样 (x, y, t) 来交叉熵loss训练另外一个神经网络来充当 $(t, x) \rightarrow p_t(y|x)$ 的classifier，这样是困难且不稳定的，因而导出了 classifier-free guidance的方法

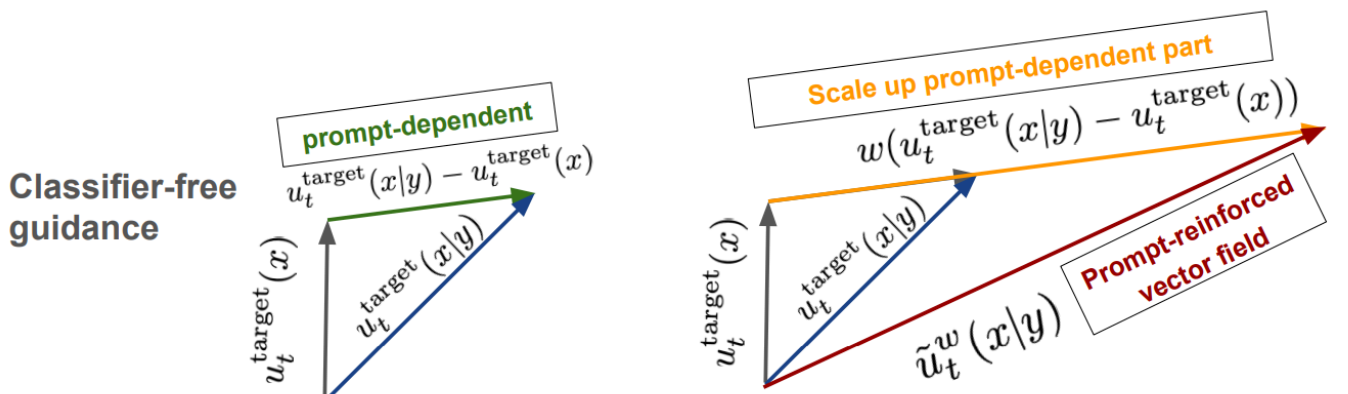
1.6.3 Classifier-Free Guidance

- 将score function表示为vector field

$$\nabla \log p_t(x|y) = \nabla \log p_t(x) + \nabla \log p_t(y|x)$$

to obtain

$$\begin{aligned} \tilde{u}_t(x|y) &= u_t^{\text{target}}(x) + w a_t \nabla \log p_t(y|x) \\ &= u_t^{\text{target}}(x) + w a_t (\nabla \log p_t(x|y) - \nabla \log p_t(x)) \\ &= u_t^{\text{target}}(x) - (w b_t x + w a_t \nabla \log p_t(x)) + (w b_t x + w a_t \nabla \log p_t(x|y)) \\ &= (1 - w) u_t^{\text{target}}(x) + w u_t^{\text{target}}(x|y). \end{aligned}$$



Remark: 作者在这里说明对于任意的条件概率路径该转换都成立，而不仅仅局限于Gaussian Probability Path

- 此处看似需要得到两种向量场，但实际上仅仅需要一个神经网络即可。注意到我们上文提到在 Guided Flow Matching中神经网络学习的是一个关于guidance $y \rightarrow u_t^{target}(\cdot|y)$ 向量场函数的映射，所以可以认为 $u_t^{target}(\cdot)$ 为输入guidance为None $y = \emptyset$ 的映射出的vector field是可以统一到该神经网络中的。
- 因而采用the absence of conditioning的trick，在训练中设定一定的概率将condition-guidance mask掉，让网络学习guidance为None $y = \emptyset$ 时对应的向量场 $u_t^{target}(\cdot)$ 。

arrive at our **CFG conditional flow matching training objective**

$$\mathcal{L}_{CFM}^{CFG}(\theta) = \mathbb{E}_{\square} \|u_t^{\theta}(x|y) - u_t^{target}(x|z)\|^2 \quad (63)$$

$$\square = (z, y) \sim p_{data}(z, y), t \sim \text{Unif}[0, 1], x \sim p_t(\cdot|z), \text{replace } y = \emptyset \text{ with prob. } \eta \quad (64)$$

Algorithm 5 Classifier-free guidance training for Gaussian probability path $p_t(x|z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d)$

Require: Paired dataset $(z, y) \sim p_{data}$, neural network u_t^{θ}

- 1: **for** each mini-batch of data **do**
- 2: Sample a data example (z, y) from the dataset.
- 3: Sample a random time $t \sim \text{Unif}_{[0,1]}$.
- 4: Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
- 5: Set $x = \alpha_t z + \beta_t \epsilon$
- 6: With probability p drop label: $y \leftarrow \emptyset$
- 7: Compute loss

$$\mathcal{L}(\theta) = \|u_t^{\theta}(x|y) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon)\|^2$$

- 8: Update the model parameters θ via gradient descent on $\mathcal{L}(\theta)$.
 - 9: **end for**
-

- 推理的时候一般将condition-guided与uncondition-guided拼接在batch维度，进行forward再加权计算合成向量场
- Classifier-Free Guidance (CFG) Flow Matching Summary

classifier-free guidance CFM (CFG-CFM) objective, given by

$$\mathcal{L}_{\text{CFM}}^{\text{CFG}}(\theta) = \mathbb{E}_{\square} \|u_t^{\theta}(x|y) - u_t^{\text{target}}(x|z)\|^2 \quad (66)$$

$$\square = (z, y) \sim p_{\text{data}}(z, y), t \sim \text{Unif}[0, 1], x \sim p_t(\cdot|z), \text{replace } y = \emptyset \text{ with prob. } \eta \quad (67)$$

In plain English, $\mathcal{L}_{\text{CFM}}^{\text{CFG}}$ might be approximated by

$(z, y) \sim p_{\text{data}}(z, y)$	► Sample (z, y) from data distribution.
$t \sim \text{Unif}[0, 1)$	► Sample t uniformly on $[0, 1)$.
$x \sim p_t(x z)$	► Sample x from the conditional probability path $p_t(x z)$.
with prob. η , $y \leftarrow \emptyset$	► Replace y with $\emptyset$ with probability η .
$\widehat{\mathcal{L}_{\text{CFM}}^{\text{CFG}}}(\theta) = \ u_t^{\theta}(x y) - u_t^{\text{target}}(x z)\ ^2$	► Regress model against conditional vector field.

At inference time, for a fixed choice of y , we may sample via

Initialization: $X_0 \sim p_{\text{init}}(x)$	► Initialize with simple distribution (such as a Gaussian)
Simulation: $dX_t = \tilde{u}_t^{\theta}(X_t y)dt$	► Simulate ODE from $t = 0$ to $t = 1$.
Samples: X_1	► Goal is for X_1 to adhere to the guiding variable y .

1.7 Neural Network Implementation of Flow Matching

1.7.1 Neural Network Architecture

从上面的分析，知道神经网络需要的输入有latent $x_t (x_0 = \epsilon)$ ，对应的时间 t ，以及condition-guidance的 y ，然后输出latent space中的vector field $u_t(x_t|y)$ 。那么对于神经网络架构的理解可以由上面三种自变量是如何输入以及因变量是如何输出展开的。

1.7.1.1 About Embedding

因为神经网络实质上是向量的运算，因而希望将设计的输入都转换为向量embedding

- Embedding time: 对于标量 t 的输入，一般会通过正余弦时间向量嵌入的方式，将其转变为向量(便于匹配神经网络的输入)

the featurization is given by

$$\text{TimeEmb}(t) = \sqrt{\frac{2}{d}} \begin{bmatrix} \cos(2\pi w_1 t) & \cdots & \cos(2\pi w_{d/2} t) & \sin(2\pi w_1 t) & \cdots & \sin(2\pi w_{d/2} t) \end{bmatrix}^T, \quad (68)$$

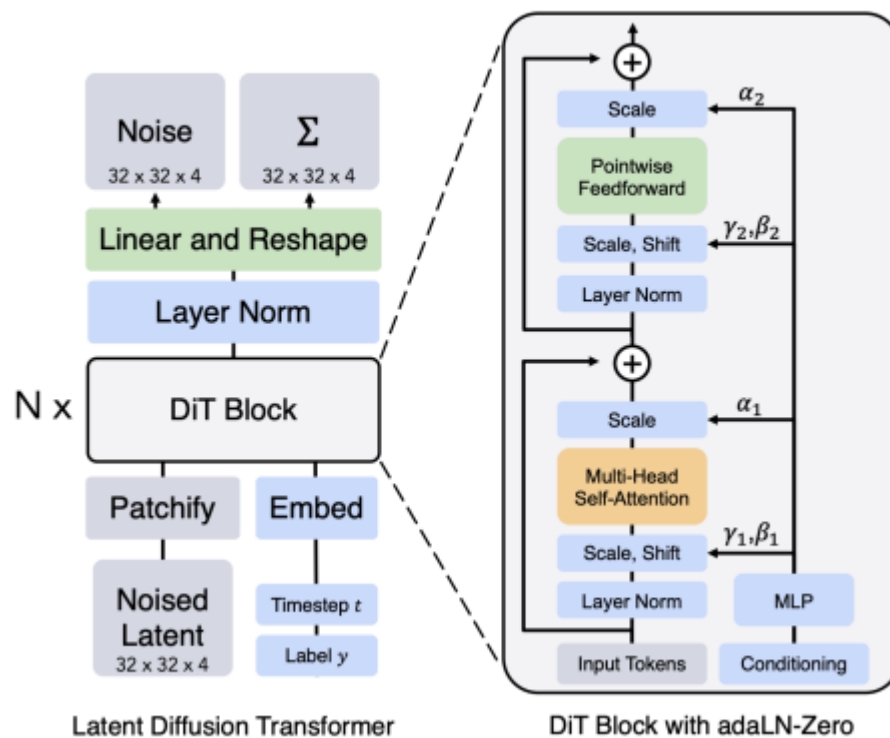
where the frequencies w_i are set in the following way

$$w_i = w_{\min} \left(\frac{w_{\max}}{w_{\min}} \right)^{\frac{i-1}{d/2-1}}, \quad i = 1, \dots, d/2. \quad (69)$$

This choice of TimeEmb is a standard choice but this exact form is not strictly necessary. Rather, the above is simply a convenient way of obtaining a normed embedding of dimension d , i.e. $\|\text{TimeEmb}(t)\| = 1$ (because $\sin^2 + \cos^2 = 1$).

- Embedding condition-guidance: 以prompt为例，会利用CLIP将language嵌入转换为embedding sequence

1.7.1.2 Diffusion Transformer (DiT)



- 利用Transformer典型架构，围绕三个输入和一个输出展开：
 - 输入的latent x 作为神经网络的输入入口
 - 输入时间 t 经过Time Embedding后，经过MLP得到AdaLN(Adaptive LayerNorm)的参数，以此注入time t
 - 输入condition-guidance y ，经过对应的Encoder(广义的encoder)后作为Block中cross-attention的Key, Value，然后将Block中的向量作为Query，以此注入guidance y
 - 在Flow Matching中是输出vector filed u ，然后作一次Euler update，再将更新后的 x 再次输入

1.7.1.3 U-Net (Before transformer)

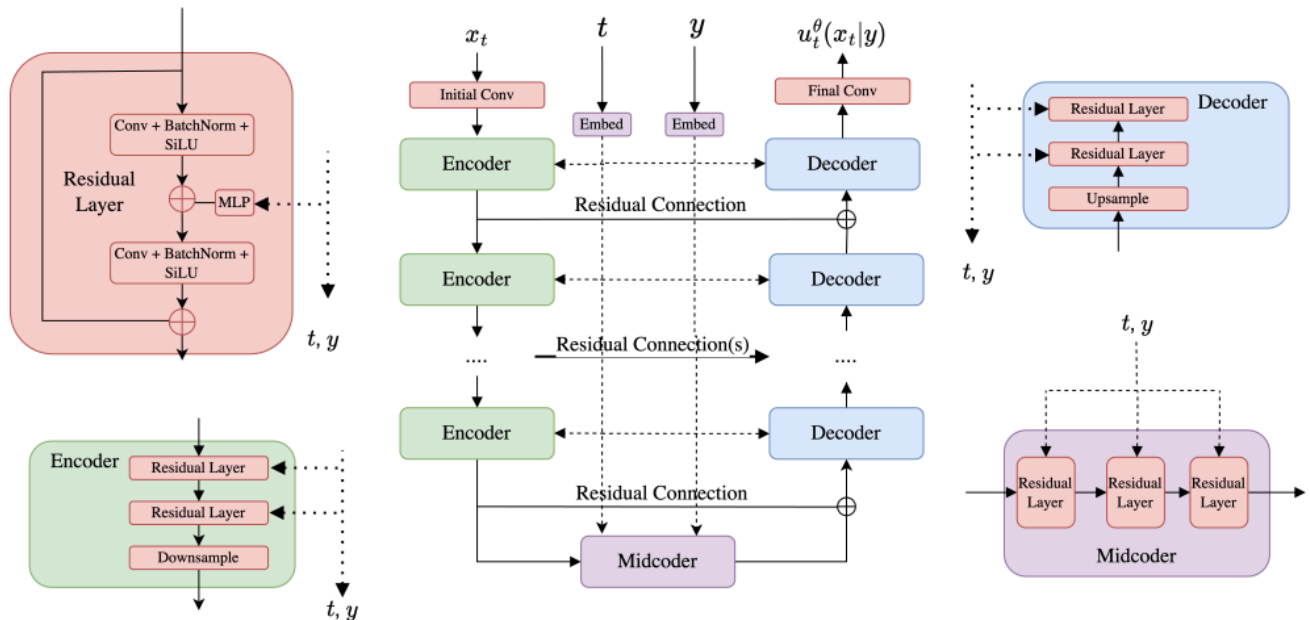


Figure 15: A simplified U-Net architecture (an architecture like this was used in lab 03 of the 2025 version of this course).

○ 围绕三个输入和一

- 输入的 x 经过Conv, 得到
- 输入的time t 和condition-guidance y 经过各自Embedding后再经MLP叠加到 x 上, 并且在每个Encoder、Midcoder和Decoder中的每层Residual Layer都再作 t 的条件输入 (可以是叠加 or AdaLN参数调整)
- 其中, 对应的Encoder的输出也残差连接到Decoder的输出
- 最后输出vector field $u_t^\theta(x_t|y)$

1.7.2 Latent Space: Need Encoder-Decoder

到目前为止, 讨论的data space $\mathbb{R}^d$, 但对于某些数据, eg image: $d = H \times W \times 3$, 这样的高维度通常是无法接受的计算代价, 因此希望将高维空间映射到一个低维空间, 在低维空间中作生成后, 再将低维空间的表征映射回高维空间得到目标生成。因此需要在Flow Matching配置一对Encoder-Decoder, 并且希望这样压缩维度的信息有损的代价对生成而言是不大的, 是可接受的 (即希望设计的Encoder-Decoder带来的compress信道有损对生成效果影响不大)

1.7.2.1 Standard Autoencoder (AE)

- 自然地, 容易想到利用两个神经网络充当Encoder-Decoder: $\mu_\phi : \mathbb{R}^d \rightarrow \mathbb{R}^k$, $\mu_\theta : \mathbb{R}^k \rightarrow \mathbb{R}^d$
- 理想地, 希望选择的 μ_ϕ 和 μ_θ 能达到高效的重建质量, 即 $\mu_\theta(\mu_\phi(x))$ resembles x on average。因此AE经常使用reconstruction loss训练:

$$\mathcal{L}_{\text{Recon}}(\phi, \theta) = \mathbb{E}_{x \sim p_{\text{data}}} [\|\mu_\theta(\mu_\phi(x)) - x\|^2].$$

- Disadvantage:

- 上述reconstruction loss训练得到的Encoder-Decoder能实现 $\mathbb{R}^d \leftrightarrow \mathbb{R}^k$ 映射, compress generation space。那么基于此对于Flow Matching, targeting the latent distribution $p_{latent}(z)$ given by $z = \mu_\phi(x), x \sim p_{data}$, then passing the output of latent generative model to decoder μ_θ for object generation。
- 虽然完成了compress压缩, 但是reconstruction loss并没有限制 $p_{latent}(z)$ "概率形状", 可能会得到到一个very hard for FM transport to learn distribution。这会影响到Flow Matching神经网络的拟合效果。

因此, 就会想要设计Encoder能将 p_{data} 映射为well-behaved and easy-to-learn p_{latent} in latent space, which is well enough to be amenable to training such a generative model (i.e., nice, simple, Gaussian-like).即希望能在latent distribution中进行regularization正则化约束, 所以引出了下文的VAE。

Remark: 因为Flow Matching一般学习的是从Gaussian Distribution到Target Distribution的transport, 因此直观上也会觉得: 如果Target Distribution也是一个Gaussian-like Distribution, 神经网络需要学习的vector field更为简单, 更容易收敛学习成功

1.7.2.2 Variational Autoencoders (VAE)

- Deterministic standard autoencoder (AE) --> Variational autoencoder: 通过放宽encoder-decoder必须是deterministic function约束, 将encoder-decoder都视作sample variant from distribution。一般常用的是Gaussian Distribution样式的Variational Encoder-Decoder, 因为其较为简单只有两个分布相关的参数"均值"与"方差"且相关后续计算方便, 只需要让神经网络输出这两个参数即可, 另外Gaussian Distribution有某种"神秘统一"的数学意义"

$$q_\phi(z|x) = \mathcal{N}(z; \mu_\phi(x), \text{diag}(\sigma_\phi^2(x))), \quad p_\theta(x|z) = \mathcal{N}(x; \mu_\theta(z), \sigma_\theta^2(z)I_d) \quad (71)$$

where $\mu_\phi(x) \in \mathbb{R}^k$, $\sigma_\phi^2(x) \in \mathbb{R}_{\geq 0}^k$, $\mu_\theta(z) \in \mathbb{R}^d$, and $\sigma_\theta^2(z) \in \mathbb{R}_{\geq 0}$ are parameterized as neural networks and diag denotes the diagonal matrix. To encode or decode a variable, we sample

$$z \sim q_\phi(\cdot|x) \quad (\text{encode})$$

$$x \sim p_\theta(\cdot|z) \quad (\text{decode})$$

Finally, we note that when $\sigma_\phi(x) = 0$ and $\sigma_\theta(x) = 0$ always, we recover a standard autoencoder. Let us examine what a reconstruction loss looks like. A natural objective is the following:

$$\mathcal{L}_{\text{VAE-Recon}}(\phi, \theta) = -\mathbb{E}_{x \sim p_{\text{data}}(x), z \sim q_\phi(\cdot|x)} [\log p_\theta(x|z)] \quad (72)$$

Remark: 变为VAE后的reconstruction loss应该从probability distribution的角度去思考定义, 因此会得到上式对数似然最大化的Loss

- 进一步的, 将VAE用comman-Gaussian-sytle实例化会得到如下VAE-reconstruction loss

$$\mathcal{L}_{\text{VAE-Recon}}(\phi, \theta) = \mathbb{E}_{x \sim p_{\text{data}}(x), z \sim q_{\phi}(z|x)} \left[\frac{1}{2\sigma_{\theta}^2(z)} \|x - \mu_{\theta}(z)\|^2 + \frac{d}{2} \log \sigma_{\theta}^2(z) \right] + \text{const} \quad (73)$$

- Additionally, in many implementations, 都是将方差项 $\sigma_{\phi}(x)$ 与 $\sigma_{\theta}(z)$ 固定为与 x, z 无关的 learned scalar constants (thereby avoiding pathological behavior and numerical stability when learning variances), 因此该VAE-reconstruction loss与AE形式基本一致。

$$\mathcal{L}_{\text{VAE-Recon}}(\phi, \theta) = \mathbb{E}_{x \sim p_{\text{data}}(x), z \sim q_{\phi}(z|x)} \left[\frac{1}{2\sigma_{\theta}^2} \|x - \mu_{\theta}(z)\|^2 \right] + \text{const} \quad (74)$$

- 回到我们想要regulation约束 p_{latent} in latent space让其nice/easy-to-learn意图, 因为VAE使用variant distribution, 而非deterministic function, 所以可以使用KL散度这一数学描述来约束latent distribution锚定与selected prior distribution的相似性。一般的是选用 $p_{\text{prior}} = \mathcal{N}(0, I_k)$ to be an isotropic Gaussian

$$\mathcal{L}_{\text{VAE-Prior}}(\phi) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [D_{\text{KL}}(q_{\phi}(\cdot|x) \parallel p_{\text{prior}})], \quad (75)$$

Remark:

- 这里约束 $q_{\phi}(\cdot|x)$ 去锚定 $p_{\text{prior}} = \mathcal{N}(0, I_k)$ 的意图前提是: "normal distribution is easy to learn, and would therefore satisfy our goal of obtaining a "trainable" latent distribution"
- 但是这里的normal distribution应该说的是 p_{latent} , 想要实现的也是对 p_{latent} 的正则约束, 这对encoder $q_{\phi}(\cdot|x)$ 的正则约束是如何对 p_{latent} 起作用的呢?
- 可以认为 $q_{\phi}(\cdot|x)$ 都是Normal Gaussian like, 再 $p_{\text{latent}} := q_{\phi}(z)$ 是从conditional到marginal的加权积分也会让 p_{latent} 也是Normal Gaussian like
- 因此结合两个不同目的的Loss, 可以定义VAE training objective Loss

$$\mathcal{L}_{\text{VAE}}(\phi, \theta) = \mathcal{L}_{\text{VAE-Recon}}(\phi, \theta) + \beta \mathcal{L}_{\text{VAE-Prior}}(\phi) \quad (78)$$

$$= -\mathbb{E}_{x \sim p_{\text{data}}(x), z \sim q_{\phi}(z|x)} [\log p_{\theta}(x | z)] + \beta \mathbb{E}_{x \sim p_{\text{data}}(x)} [D_{\text{KL}}(q_{\phi}(\cdot|x) \parallel p_{\text{prior}})] \quad (79)$$

- 进一步的, 将VAE用comman-Gaussian-sytle实例化散度 D_{KL} 可以得到:

$$\mathcal{L}_{\text{VAE-Prior}}(\phi) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [D_{\text{KL}}(q_{\phi}(\cdot|x) \parallel \mathcal{N}(0, I_k))] = \mathbb{E} \left[\frac{1}{2} \mathcal{K}(\sigma_{\phi}^2(x)) + \frac{1}{2} \|\mu_{\phi}(x)\|^2 \right] \quad (82)$$

- $\mathcal{K}$ 定义与证明如下:

Example 31 (KL Divergence Between Isotropic Gaussians)

Let $q(x) = \mathcal{N}(x; \mu_q, \text{diag}(\sigma_q^2))$ and $p(x) = \mathcal{N}(x; \mu_p, \text{diag}(\sigma_p^2))$ be Gaussians with diagonal covariance matrices, with $\sigma_q, \sigma_p \in \mathbb{R}_{\geq 0}^d$, and where $x \in \mathbb{R}^d$. Then

$$D_{\text{KL}}(q \parallel p) = \frac{1}{2} \left(\mathcal{K} \left(\frac{\sigma_q^2}{\sigma_p^2} \right) + \frac{\|\mu_q - \mu_p\|^2}{\sigma_p^2} \right), \quad \text{where } \mathcal{K}(\alpha) = \sum_{i=1}^d \alpha_i - \log \alpha_i - 1. \quad (80)$$

The expression above is intuitive: If the mean and variances coincide, that then $D_{\text{KL}}(q \parallel p) = 0$. Further, it increases with the squared error $\|\mu_q - \mu_p\|^2$ between the mean vectors. Finally, the function $\mathcal{K}(\alpha)$ has a unique minimum at $\alpha = 1$ so that $D_{\text{KL}}(q \parallel p)$ is minimized when $\sigma_q = \sigma_p$.

Proof. We do the proof for $d = 1$ (proof is analogous for $d > 1$ by summing up each dimension). Given the density of the normal distribution, we know that (see Equation (97)):

$$\log q(x) = -\frac{1}{2} \log(2\pi\sigma_q^2) - \frac{1}{2\sigma_q^2} \|x - \mu_q\|^2, \quad \log p(x) = -\frac{1}{2} \log(2\pi\sigma_p^2) - \frac{1}{2\sigma_p^2} \|x - \mu_p\|^2$$

Then

$$D_{\text{KL}}(q \parallel p) = \mathbb{E}_{x \sim q} [\log q(x) - \log p(x)] = \frac{1}{2} \log \frac{\sigma_p^2}{\sigma_q^2} + \frac{1}{2\sigma_p^2} \mathbb{E}_q [\|x - \mu_p\|^2] - \frac{1}{2\sigma_q^2} \mathbb{E}_q [\|x - \mu_q\|^2]. \quad (81)$$

For $x \sim \mathcal{N}(\mu_q, \sigma_q^2 I)$ we have

$$\mathbb{E}_q [\|x - \mu_q\|^2] = \text{tr}(\sigma_q^2 I) = \sigma_q^2.$$

Combining this with the fact that $x - \mu_p = (x - \mu_q) + (\mu_q - \mu_p)$, and $\mathbb{E}_q[x - \mu_q] = 0$, we obtain

$$\mathbb{E}_q [\|x - \mu_p\|^2] = \mathbb{E}_q [\|x - \mu_q\|^2] + \|\mu_q - \mu_p\|^2 = \sigma_q^2 + \|\mu_q - \mu_p\|^2.$$

Plugging these into Equation (81) yields (80). □

- 最终对于comman-Gaussian-sytle的VAE的total loss如下

$$\begin{aligned} \mathcal{L}_{\text{VAE}}(\phi, \theta) &= \mathcal{L}_{\text{VAE-Recon}}(\phi, \theta) + \beta \mathcal{L}_{\text{VAE-Prior}}(\phi) \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x), z \sim q_{\phi}(z|x)} \left[\underbrace{\frac{1}{2\sigma_{\theta}^2(z)} \|x - \mu_{\theta}(z)\|^2}_{\text{recon. error}} + \underbrace{\frac{d}{2} \log \sigma_{\theta}^2(z)}_{\text{decoder confidence}} + \underbrace{\frac{\beta}{2} \mathcal{K}(\sigma_{\phi}^2(x))}_{\text{make latent variance}=1} + \underbrace{\frac{\beta}{2} \|\mu_{\phi}(x)\|^2}_{\text{make latent mean}=0} \right] \end{aligned} \quad (83)$$

1.7.2.3 Training a VAE Implementation Detail

- 从上面的 $\mathcal{L}_{\text{VAE}}(\phi, \theta)$ 是无法训练的，因为 $\mathbb{E}$ 有sample $z \sim q_{\phi}(z|x)$ still depends on the parameter ϕ ，gradient无法通过采样反传到 ϕ 参数且也不适配主流训练infra，所以采用重参数化技巧(reparameterization trick):

$$q_\phi(z|x) = \mathcal{N}(z; \mu_\phi(x), \sigma_\phi^2(x)I_k)$$

we can obtain samples via

$$\epsilon \sim \mathcal{N}(0, I_k), \quad z = \mu_\phi(x) + \sigma_\phi(x)\epsilon \quad \Rightarrow \quad z \sim q_\phi(\cdot|x)$$

Note that in this equation, the only source of noise/stochasticity is from ϵ whose distribution is independent of ϕ . Therefore, we can rewrite the loss as:

$$\mathcal{L}_{\text{VAE}}(\phi, \theta) = \mathbb{E}_{x \sim p_{\text{data}}(x), \epsilon \sim \mathcal{N}(0, I_k)} \left[\frac{1}{2\sigma_\theta^2(z)} \|x - \mu_\theta(\mu_\phi(x) + \sigma_\phi(x)\epsilon)\|^2 + \frac{d}{2} \log \sigma_\theta^2(z) + \frac{\beta}{2} \mathcal{K}(\sigma_\phi^2(x)) + \frac{\beta}{2} \|\mu_\phi(x)\|^2 \right]$$

After reparameterization, the randomness comes only from $\epsilon \sim \mathcal{N}(0, I_k)$, whose distribution does not depend on ϕ . Therefore, we can minimize this loss with the standard tools of deep learning. To simplify things even further, we can set $\sigma_\theta^2(z) = \sigma^2$ constant everywhere again and obtain:

$$\mathcal{L}_{\text{VAE}}(\phi, \theta) = \mathbb{E}_{x \sim p_{\text{data}}(x), \epsilon \sim \mathcal{N}(0, I_k)} \left[\frac{1}{2\sigma^2} \|x - \mu_\theta(\mu_\phi(x) + \sigma_\phi(x)\epsilon)\|^2 + \frac{\beta}{2} \mathcal{K}(\sigma_\phi^2(x)) + \frac{\beta}{2} \|\mu_\phi(x)\|^2 \right]$$

In [Algorithm 6](#), we summarize the training procedure of the VAE.

◦ Training VAE Algorithm:

Algorithm 6 β -VAE Training Procedure (Gaussian decoder with fixed variance $p_\theta(x|z) = \mathcal{N}(x; \mu_\theta(z), \tilde{\sigma}^2 I_d)$)

Require: Dataset of samples $x \sim p_{\text{data}}$, encoder networks $(\mu_\phi(x), \log \sigma_\phi^2(x))$, decoder network $\mu_\theta(z)$, latent dim k , constants $\beta \geq 0$, $\sigma^2 > 0$

- 1: **for** each mini-batch $\{x_i\}_{i=1}^B$ **do**
- 2: Encode each x_i : $\mu_i \leftarrow \mu_\phi(x_i)$, $\log \sigma_i^2 \leftarrow \log \sigma_\phi^2(x_i)$
- 3: Sample noise $\epsilon_i \sim \mathcal{N}(0, I_k)$
- 4: Reparameterize: $z_i \leftarrow \mu_i + \sigma_i \odot \epsilon_i$ (where $\sigma_i = \exp(\frac{1}{2} \log \sigma_i^2)$)
- 5: Decode mean: $\hat{x}_i \leftarrow \mu_\theta(z_i)$
- 6: Reconstruction loss:

$$\mathcal{L}_{\text{recon}} \leftarrow \frac{1}{B} \sum_{i=1}^B \frac{1}{2\tilde{\sigma}^2} \|x_i - \hat{x}_i\|^2$$

- 7: KL loss to the prior $p_{\text{prior}}(z) = \mathcal{N}(0, I_k)$:

$$\mathcal{L}_{\text{KL}} \leftarrow \frac{1}{B} \sum_{i=1}^B \frac{1}{2} \sum_{j=1}^k (\mu_{i,j}^2 + \sigma_{i,j}^2 - \log \sigma_{i,j}^2 - 1)$$

- 8: Total loss: $\mathcal{L} \leftarrow \mathcal{L}_{\text{recon}} + \beta \mathcal{L}_{\text{KL}}$
 - 9: Update $(\phi, \theta) \leftarrow \text{grad_update}(\mathcal{L})$
 - 10: **end for**
-

◦ 这里再给出一些Note作者的Practical remarks:

1. **Choosing β (and KL warm-up).** Large β enforces latents closer to the prior but can hurt reconstructions and may trigger *posterior collapse* (the encoder ignores x and outputs $q_\phi(z|x) \approx \mathcal{N}(0, I_k)$). A common stabilization is *KL warm-up*: start with $\beta = 0$ and gradually increase it to a target value over the first epochs. However, in all modern autoencoders, the β value is very small, i.e. $\beta \ll 1$.

2. **Decoder variance.** Learning a Gaussian decoder variance σ_θ^2 can be numerically delicate and may lead to degenerate solutions unless regularized. For stability, many implementations fix $p_\theta(x|z) = \mathcal{N}(x; \mu_\theta(z), \sigma^2 I_d)$ with constant σ^2 , which makes the reconstruction term proportional to mean-squared error (up to constants).
3. **Reconstruction losses beyond pixel MSE.** For images, a pixelwise Gaussian likelihood (mean squared error) often yields overly smooth reconstructions. In practice, people add *perceptual losses* (feature-space losses using a pretrained network) to improve sharpness and semantic fidelity.
4. **Adversarial and hybrid objectives.** To further improve visual realism, one can combine the VAE objective with an *adversarial loss* (VAE-GAN style), using a discriminator on decoded samples. This typically sharpens outputs but introduces additional optimization instability and extra hyperparameters.

Remark: TODO  用户191488

- "3."谈及的就是关于reconstruction loss在latent space作还是data space作的对比。在robot action generation中是否也有不同的观点呢？（pi05是在action space作reconstruction loss）
- "4."了解GAN  用户191488
- 原Note作者在Remark中谈及到目前Flow Matching image generative model's performance可能极大程度上受VAE影响，**Representation is worthwhile !**

1.7.2.4 Action Space to Latent Space

1.7.2.4.1 Image generative model VS action generative model

- 降维 vs 升维
 - image generative model 通过VAE实现从image space到latent space的**降维**转换，
 - 在action generative model中一般是通过MLP实现action_in_project与action_out_project实现从action space到latent space的**升维**转换
- Flow Matching Loss 的作用空间
 - Image generative model 的 flow matching loss 一般是在latent space进行计算监督训练
 - Action generative model 的 flow matching loss 一般是在latent space denoise后 action_out_project到action space进行计算监督
- Encoder-Decoder的训练Loss
 - Image generative model 的 VAE 是有自己的 $\mathcal{L}_{VAE}$ 监督 VAE 即能完成reconstruction compress又可以正则约束 p_{latent} in latent space 为easy-to-learn Gaussian like
 - Action generative model 的 action_in/out_project 是根据flow matching loss 联合训练更新参数的

1.7.2.4.2 Why Action Space to Latent Space

- 本人认为基本从表征学习的角度理解更合适

- i. action 提到 latent 上能够与 vision 或者 language 那些高维特征进行良好交互，而如果不提则很难做到，可能出现特征不匹配mismatch
- ii. 即使不确定 action space 对于flow matching生成好不好，给他一个学习投影层的选择，就能够自己找到比较合适的表征，而不用人为设计（也未必更好），这里的观点是直接通过最终action generation的目标来引导action_in/out_project网络找到最适合这个目标的表征（绝对的目标导向性引导网络，而忽略人为对表征的设计）。[这里还是相信吴恩达在CS229: Machine Learning中提到的“良好合理的假设设计先验可以减小模型学习的burden，也许能获得更加的表现”，在完全相信深度学习的浪潮下，是否可以认为某些auxiliary loss的设计是一种在human assumption design和deep learning最终绝对目标导向的一种comprise]

◦ 以下给出  李嘉良 关于此问题的回复原文供学习参看，万分感谢

我觉得有几个原因：

1. 研究惯性：之前flow matching用于生图是在latent space下的，因为pixel space太大了（如果是video generation的话更大，需要用 VAE 额外压缩时间维度），所以用到action这边也顺便在latent space下做了；

2. 对 VAE 的认知：VAE 这边你可以把他当做是一个转换器，确实那些from scratch的image generation会去训VAE，但之后 VAE 作为一个良好的转换器，在基于这个image model上的finetuning（同样 flow matching）上一般是不会训的，甚至有些工作会拿这个 VAE 作为自己 from scratch 网络的数据转换器。如果从这个角度上看 VAE 与 flow matching 想要优化的问题本身未必会是强耦合的；

3. action space与latent space的关系：（1）从工程角度上讲，action 提到 latent 上能够与 vision 或者 language 那些高维特征进行良好交互，而如果不提则很难做到，除非你将 action 也视为 language 去 tokenize，但是用文本来表示动作在连续-离散上未必考虑周到（当然不是说不好），而且从来源性来讲动作都是一个独立的模态，不视为 language 去 tokenize也许也合理。但即便你将他视作 language 去 tokenize，也算是提到高维了；（2）从可能比较理论的角度上讲，action space 本身是存在着一些约束的，在分布上未必有 latent space 那样“平滑规则”，他的 manifold 也可能比较麻烦，但是 latent space 本身会根据 action 投影层的参数更新去优化，是能够隐式且数据驱动式地找到一个对模型比较好的表示。而且我不确定 action 本身带来的一些噪声会不会对 action 学习有影响，但是如果投影的话能够数据驱动式地忽略这个点，latent space 的高维度说不定能捕捉到低维 action 表示不出来的信息，比如一些特殊时空相关性之类的；（3）假如追求所谓的机器人多构型，或者完成任务的多模态表示（一个任务的多种可能完成方式），那可能也是到 latent 会更好，而 action 本身会比较难做到这一点；（4）从表征学习的角度来看，就算我们不确定 action space 对于模型好不好，给他一个学习投影层的选择，就能够自己找到比较合适的表征，而不用人为设计（也未必好），说不定哪怕我们给他的 latent 维度不是很高也能找到比较适合的表征。而且从 AI 的发展来看，你甚至也可以将以前那种老的 image classification 的卷积网络也视为特征提取器，最终是给分类头好的输入以判断的，哪怕他们的image输入可能也就 32x32x3，从这个角度看也是为了提取较好的表征。